SEASUG Paper 79-2026

Zero-Friction R for GxP: A Centralized Infrastructure Bridging SAS® and R in Clinical Programming

Jaime Yan, George Wu, and Jessie Yan, Kardigan

ABSTRACT

Most statistical programming departments do not start fresh. When R is added to an established SAS®-based clinical programming environment on a shared, locked-down Windows server, R programs must inherit folder conventions, batch schedulers, and validation expectations designed for SAS. Published accounts mostly describe greenfield R platforms or full SAS-to-R migration; the coexistence path is far less documented.

We present the R equivalent of SAS's autopath.sas: a centralized configuration system that detects each program's position in the compound/study/milestone hierarchy, derives data and output paths, loads a version-locked package library, and resolves namespace conflicts — all automatically at startup. IT installs one three-line site file once; maintenance lives in a team-managed configuration directory, and new users configure nothing.

Core techniques: (1) redirecting renv's project-scoped library to a single server-level, version-pinned library shared by 25 study folders across five compounds, with append-only versioned libraries so completed studies re-run reproducibly; (2) a guard that restores utility functions after legacy rm(list=ls()) calls; (3) identical environments in interactive sessions and scheduled batch jobs; and (4) package loading adapted to the program type inferred from the folder structure.

Deployed across five compounds, the system replaced a documented 30–60 minute per-user setup checklist with a single automatic startup, while retaining GxP-compatible controls: locked package versions, enforced R versions, and audit-trail support. The approach transfers to any organization adding R capability alongside existing SAS infrastructure.

1. INTRODUCTION

1.1 THE INDUSTRY TRANSITION

Statistical computing in the pharmaceutical industry is in the middle of a gradual platform transition. SAS has been the dominant environment for clinical trial analysis and regulatory submission for three decades, but R adoption has accelerated on the strength of three developments. First, the FDA's 2015 Statistical Software Clarifying Statement made explicit that the agency "does not require use of any specific software for statistical analyses," removing the perceived regulatory barrier to R-based submissions (FDA 2015). Second, the pharmaverse ecosystem now provides a coherent, clinically focused package stack — admiral for ADaM derivation, rtables and r2rtf for reporting, and many more — that covers the end-to-end clinical programming workflow. Third, the R Validation Hub's white paper has given organizations a risk-based vocabulary for assessing R packages, replacing the earlier all-or-nothing anxiety about open-source validation (R Validation Hub).

None of these developments, however, tells a programming team *how to wire R into the building it already lives in*.

1.2 THE PRACTICAL GAP: COEXISTENCE, NOT GREENFIELD OR MIGRATION

Published accounts of R adoption in our industry mostly describe one of two scenarios: greenfield R platforms, where a team builds a new environment from scratch with modern tooling; or full SAS-to-R migration, where R replaces SAS outright (see, for example, Zhang and Cook 2025 and Bowsher et al. 2021 for workflow- and reproducibility-focused accounts, and Lee and Jeeva 2022 for a department-scale migration). Our reading of recent PharmaSUG, PHUSE, and SEASUG proceedings suggests a third

scenario is far less documented: the coexistence path, in which R is added *alongside* an intact, validated SAS installation on a shared, locked-down Windows server, and R programs must inherit folder conventions, batch schedulers, and validation expectations that were designed for SAS. We are not alone in this reading: Palukuru et al. (2022), describing R as a validation tool alongside SAS, explicitly note "a dearth of literature exploring concurrent use" of the two systems. To be precise, we are not aware of published accounts of this coexistence path with operational detail — how paths, packages, batch parity, and user onboarding are actually handled when R must behave like a well-mannered guest in a SAS house.

This paper is such an account.

1.3 HOW SAS ACHIEVES ZERO FRICTION IN OUR STUDIO

Our organization's clinical programmers work in a studio application that manages the full lifecycle of clinical TFL (table, figure, listing) programming. The studio enforces a standardized folder hierarchy across all studies: a workspace contains compounds, each compound contains studies, and each study contains milestone folders, one per data snapshot:

```
workspace/
├── <Compound>/
│   └── <Study>/
│       └── <Milestone>/            e.g., one interim or final data
snapshot
│           ├── Programs/
│           │   ├── Table/
│           │   ├── Figure/
│           │   ├── Listing/
│           │   ├── ADaM/
│           │   └── SDTM/
│           ├── Data/
│           │   ├── Raw/
│           │   ├── SDTM/
│           │   ├── ADaM/
│           │   └── Other/
│           ├── Output/
│           ├── Documents/
│           └── Utilities/
├── Global Macro Library/           ← SAS convention stack
└── R_Config/                       ← NEW: centralized R configuration
(this paper)
```

The workspace holds five production compounds — six including the internal test tree — comprising 25 study folders and 63 milestone folders with program directories (folder enumeration on the production server, 2026-09-28; no data files were read).

Every SAS program in this environment begins with the same two lines, inserted automatically by the program templates:

```
%include "<shared network location>/Global Macro Library/autopath.sas";
%autopath;
```

The `autopath.sas` macro detects the calling program's full path, walks up the directory tree to identify compound, study, and milestone, assigns SAS libraries (`libname raw "..."; libname adam "...";`), and sets macro variables such as `&cmpd`, `&study`, and `&tflpath`. A companion `setup.sas` applies standard options. This is what we mean by *zero friction*: a programmer opens a program file, and paths, libraries, and conventions are already correct. Nobody configures anything by hand, and no per-study copies of the convention stack exist — there is one `autopath.sas`, centrally maintained.

1.4 THE R GAP

When we began adding R capability to the studio, the contrast with SAS was stark. Table 1 summarizes what SAS programmers take for granted and what R had in our environment before this work.

Table 1. The SAS-to-R capability gap before this work.

SAS capability (studio-provided)	R equivalent before this work
`autopath.sas` — path auto-detection from program location	None — paths hardcoded per program
`setup.sas` — library assignment and standard options	None — manual `setwd()` and ad hoc options
Centralized macro library, one copy for all studies	None — packages installed ad hoc per user
Locked, uniform software version for all users	None — users ran whatever R and packages they had installed
Template auto-population of standard header and setup lines	Partial — minimal templates existed
Batch submission through the studio scheduler	Partial — a batch submission script existed but ran with no environment setup
Batch/interactive parity (same environment either way)	None — interactive and batch sessions differed

1.5 GOAL

Our goal was narrow and concrete: replicate the `autopath.sas` zero-friction experience for R, within the same studio platform, using the same folder structure, on the same shared Windows server, with the same batch scheduler — without requiring ongoing administrator access, and without asking programmers to configure anything by hand.

1.6 COMPANION PAPER

This paper describes the infrastructure layer only: how any R program in the studio acquires paths, packages, and conventions automatically. The reporting application built on top of this infrastructure — a convention-driven system in which one R code path produces submission-ready TLFs in both scheduled batch production and interactive preview — appears in our companion paper, “One Code, Two Modes: A Convention-Driven R Reporting System for Submission-Ready Clinical TLFs” (same authors, SEASUG 2026).

---

2. ARCHITECTURE

2.1 DESIGN PRINCIPLES

Four design principles governed every decision:

Zero-friction user experience. Programmers should experience the same ease as SAS: open a file, run code, paths work. Any design that required per-user or per-study configuration was rejected, however elegant.

Single source of truth. All configuration lives in one location. No copies of setup logic in study folders — the same reason there is exactly one `autopath.sas`.

Constraint-compatible. The design must work on a shared Windows server without administrator access for the programming team, without write access to the workspace root, and with the studio’s legacy batch scheduler.

GxP-compatible controls. Package versions locked, R version enforced, behavior reproducible across time, and execution auditable — controls that support operation in a GxP context. We deliberately say *GxP-compatible controls*, not GxP compliance; Section 4.4 states exactly what is and is not claimed.

2.2 THE R_CONFIG DIRECTORY

All R configuration lives in a single writable directory, `R_Config/`, sitting beside the Global Macro Library at the workspace root:

```
<shared network location>/workspace/R_Config/
├── .Renviron          → Environment variables for every studio R session
├── .Rprofile          → Package loading, options, conflict resolution,
banner
├── rconfig.R          → Path detection + setup() function (≈ autopath.sas)
├── init_renv.R        → One-time package library initialization
├── test_rconfig.R     → Verification suite run after any configuration
change
├── sync_r_config.ps1  → Bidirectional sync with the development
environment
└── renv/              → renv activation script and versioned package
libraries
```

The full inventory, with file sizes and responsibilities, is given in Appendix A. The programming team has write access to this directory; IT does not need to be involved after the one-time installation described next.

2.3 THE THREE-LAYER BOOTSTRAP

The loading chain has three layers (Figure 1):

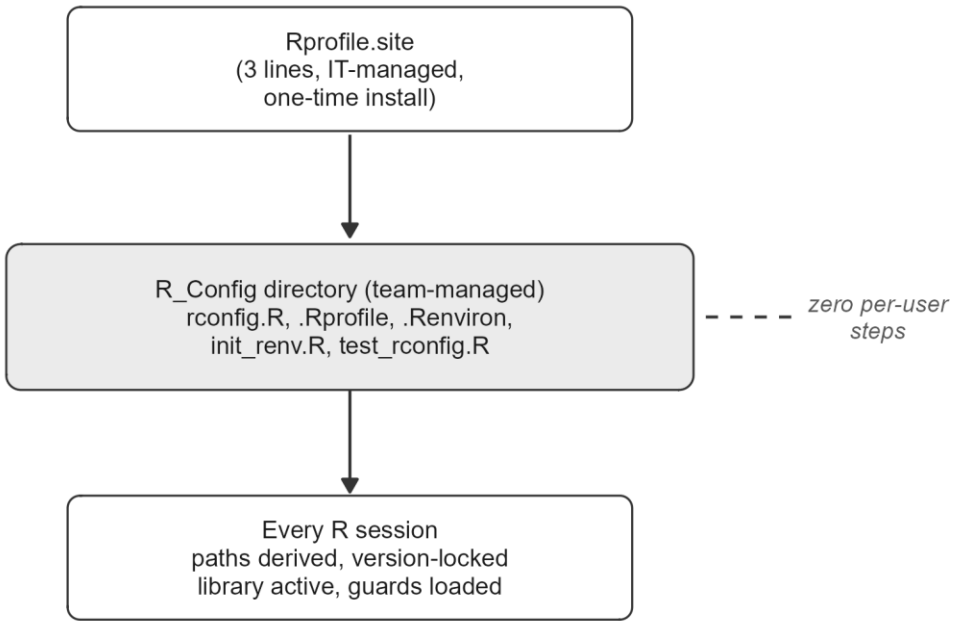


Figure 1. Three-layer bootstrap chain: one IT-owned site file points at a team-managed configuration directory; every session inherits the result.

We want to be explicit about what this is and is not. The three-layer bootstrap is **not a novel mechanism**: `Rprofile.site`, user `.Rprofile` files, and the environment variables that point R at them are

documented R startup facilities, described in the “Initialization at Start of an R Session” section of *R Installation and Administration* (R Core Team). The contribution here is the *packaging*: using the site profile as a permanent, minimal pointer to a team-managed configuration directory, so that IT’s involvement ends after one three-line file. The entire site file is:

```
# Rprofile.site — installed once by IT in the R installation's etc/ folder;
the only admin-owned file in this design.
rconfig <- "<shared network location>/R_Config/rconfig.R"
if (file.exists(rconfig)) source(rconfig)
```

Every R session on the server — any user, interactive or batch — sources `rconfig.R` at startup (Layer 2), which defines `setup()` and the path-detection machinery and then bootstraps `.Rprofile` (Layer 3), which activates the locked package library, loads packages, resolves namespace conflicts, and prints an environment banner. Because Layers 2 and 3 live in the writable `R_Config/` directory, all later maintenance — new packages, version bumps, banner changes — is a team action, not an IT ticket.

2.4 EXECUTION FLOWS: INTERACTIVE AND BATCH

Figure 2 contrasts the interactive and batch entry points as they pass through the shared bootstrap.

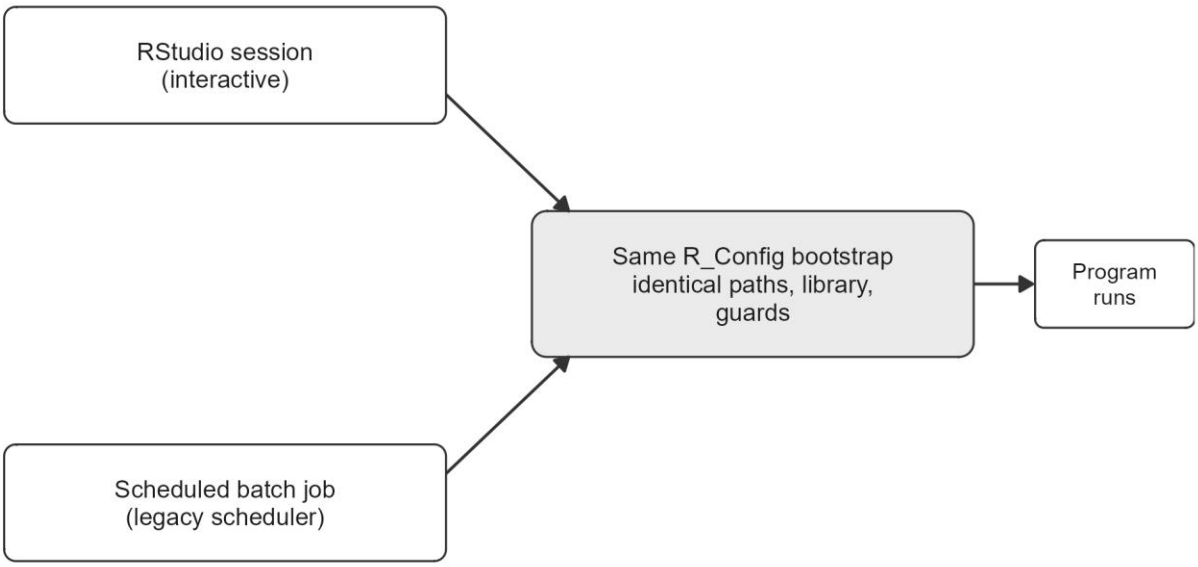


Figure 2. Interactive vs. batch execution flow: two entry points, one bootstrap, one resulting environment.

Interactive use is the simple case: the user opens any R file under a milestone’s `Programs/` tree, R starts, the bootstrap chain runs, and `setup()` locates the program through the RStudio API. Batch use required deliberate engineering, because a scheduled `Rscript` process has no RStudio session and no inherited user profile. The studio’s batch wrapper therefore injects a small set of environment variables — a pointer to the program file being run, and pointers to the central `.Rprofile` and `.Renviron` — before invoking `Rscript`. The result is the property we cared most about: **batch and interactive sessions are the same environment**, produced by the same code path, differing only in how the program’s location is discovered (Section 3.1).

2.5 COMPARISON LANDSCAPE

Several published or productized approaches address overlapping parts of this problem. We position our work against each honestly.

envsetup (pharmaverse). The envsetup package provides project-environment setup for R-based clinical workflows: configurable paths (data, output, programs) and automatic sourcing of R scripts from specified directories, driven by a YAML configuration. It is the closest published relative of our `rconfig.R`. The feature comparison below was verified against envsetup 0.3.0 (current CRAN release), documentation accessed 2026-09-26.

Capability	envsetup 0.3.0	This paper (R_Config)
Auto-load functions/data/packages at session start	Yes (via a call placed in the project `.Rprofile`)	Yes
Path derivation from compound/study/milestone folder position	No (YAML project-config based)	Yes (the R `autopath`)
Shared server-level library across 25 study folders	Not its target	Yes (Section 3.4–3.5)
Program-type-aware package loading from folder name	No	Yes (Section 3.6)
Batch/interactive parity via scheduler injection	No	Yes (Section 3.8)
Admin footprint	None (per-project configuration)	One-time site file by IT, then none

The honest summary: envsetup is a well-engineered solution for R-centric project structures; our constraint set — an existing SAS folder hierarchy that R must conform to, rather than a structure R gets to define — led us to a configuration directory instead.

Posit Package Manager (PPM). PPM serves dated CRAN snapshots, giving centralized, locked package repositories across many projects — directly comparable to our versioned central libraries (Section 3.5), and the stronger choice where the infrastructure is available. It requires server infrastructure and licensing that our environment did not have; our approach approximates “dated snapshots” with append-only versioned library directories on a file share.

pkgr + lockfiles. pkgr installs packages from a lockfile into a specified library and is a proven route to shared, validated libraries. It addresses package installation, not the path-detection, template, and scheduler-parity layers that occupied most of our effort; the two are complementary rather than competing, and a pkgr-managed library could sit underneath our bootstrap unchanged.

Containers. Container images (Docker, or Apptainer on HPC) give immutable, fully reproducible environments and are the right answer where container infrastructure exists. Our constraint was a shared Windows server with a legacy batch scheduler and minimal admin rights; there was no container runtime, and introducing one was not within the programming team’s authority.

rig. rig manages multiple R installations and versions cleanly. Our governance model instead mandates a single R version per server (Section 4.2), deliberately mirroring the SAS model of one validated version for all users; where multiple R versions are legitimate, rig is the better tool.

Positioning. Under our constraints — shared Windows server, minimal admin rights, legacy batch scheduler, an existing SAS folder convention, and GxP-compatible control requirements — a plain configuration directory plus a three-line site file was the smallest mechanism that satisfied every constraint. We offer it as a pattern for teams in the same corner, not as a replacement for the better-tooled alternatives above.

3. TECHNICAL IMPLEMENTATION

This section follows the outline's code-exhibit policy: we show the three exhibits that carry the design — the site file (already shown in Section 2.3), the `setup()` path-derivation core, and the dual-check guard — and refer everything else to the configuration-directory inventory in Appendix A.

3.1 PATH DETECTION CASCADE

Everything downstream depends on one question: *where is the program that is running?* R has no equivalent of SAS's automatic `&sysin`, so `rconfig.R` implements a four-method cascade, tried in order:

Batch environment variable. The studio's batch wrapper injects the full path of the program being executed. If present, this wins — batch mode is the most controlled context.

RStudio active document. Via the `rstudioapi` package, the path of the file open in the editor. This covers interactive work.

`Rscript --file=`. Parsing `commandArgs()` covers command-line invocation outside the studio scheduler.

`source()` call frames. Walking `sys.frames()` for an `ofile` attribute covers programs reached indirectly through `source()`.

If all four fail, startup stops with an explicit error rather than guessing — a wrong path in clinical programming is worse than no path. The full function (`rconfig.R`, 462 lines total; Appendix A) additionally normalizes path separators and validates that the resolved path sits inside the workspace hierarchy.

3.2 PATH DERIVATION: SETUP() — THE R AUTOPATH

Once the program path is known, `setup()` walks up the directory tree using the same algorithm as SAS's `autopath`. This is the heart of the paper, so we show it in full (de-identified):

```
setup <- function(pgmpath = NULL, ...) {
  if (is.null(pgmpath)) pgmpath <- .get_current_program_path()

  # Walk up: program.R -> Table -> Programs -> milestone -> study ->
compound -> root
  pgmdir       <- dirname(pgmpath)
  pgmtyp       <- toupper(basename(pgmdir))      # e.g., "TABLE"
  programs_dir <- dirname(pgmdir)                # "Programs"
  snapdir      <- dirname(programs_dir)          # milestone (data snapshot)
  studydir     <- dirname(snapdir)               # study
  cmpddir      <- dirname(studydir)              # compound

  # Data library paths (the SAS libname equivalents)
  rawdata  <- file.path(snapdir, "Data", "Raw")
  sdtmdata <- file.path(snapdir, "Data", "SDTM")
  adamdata <- file.path(snapdir, "Data", "ADaM")

  # Return a named list (the R equivalent of SAS macro variables +
libnames)
  bps <- list(
    cmpd = basename(cmpddir), study = basename(studydir),
    snap = basename(snapdir), pgmtyp = pgmtyp,
    lib = list(raw = rawdata, sdtm = sdtmdata, adam = adamdata),
    tflpath = file.path(snapdir, "Output")
  )
  invisible(bps)
}
```

The returned object gives SAS programmers a familiar mapping:

SAS	R (this implementation)
`&cmpd`, `&study`, `&snap`	`bps$cmpd`, `bps$study`, `bps$snap`
`libname raw "..."`	`bps$lib$raw`
`libname adam "..."`	`bps$lib$adam`
`&tflpath`	`bps$tflpath`
`data raw.dm; set raw.dm;`	`haven::read_sas(file.path(bps$lib$raw, "dm.sas7bdat"))`

Program templates contain exactly two lines — `source("<R_Config>/rconfig.R")` and `bps <- setup()` — the direct R analogue of the `%include` / `%autopath` pair, and equally invisible in daily use because the site file has usually already sourced `rconfig.R` before the template line runs (the call is idempotent).

3.3 THE RM(LIST=LS()) PROBLEM AND THE DUAL-CHECK GUARD

A large fraction of legacy R templates and training materials open with `rm(list=ls())` to guarantee a clean workspace. In our design that habit is destructive: it wipes not only the user's objects but also the utility functions that `.Rprofile` just loaded — silently, mid-session, after startup appeared to succeed.

We present our solution as a **defensive pattern**, not an innovation. It rests on one observation: `options()` are stored in R's internal state, *not* in the global environment, so they survive `rm(list=ls())`. The guard therefore checks two things — an option flag recording that `.Rprofile` ran, and the actual existence of a known utility function:

```
# In rconfig.R, before setup() proceeds:
if (!isTRUE(getOption("studio.rprofile.loaded")) ||
    !exists("studio_load", mode = "function")) {
  source(file.path(Sys.getenv("STUDIO_R_CONFIG"), ".Rprofile"))  # restore
what rm() wiped
}

# In .Rprofile, the mirror-image guard prevents double initialization:
if (isTRUE(getOption("studio.rprofile.loaded")) &&
    exists("studio_load", mode = "function")) {
  # already fully loaded — skip
} else {
  # ... full initialization, then:
  options(studio.rprofile.loaded = TRUE)
}
```

Either check alone has a failure mode; together they cover the cases we have observed in production:

Failure mode	Option flag says	Functions present?	Guard behavior
User program calls `rm(list=ls())` mid-session	"loaded" (options survive)	No (wiped)	Function check fails → `.Rprofile` re-sourced, utilities restored
Conflicting `options()` reset the flag (e.g., another script clears options)	Cleared	Yes	Flag check fails → `.Rprofile` re-sourced; idempotency guard makes this harmless

Failure mode	Option flag says	Functions present?	Guard behavior
renv autoloader disabled in a project (escape hatch, Section 3.4)	"loaded"	Yes	Both checks pass → no reload; session proceeds with project-local settings by explicit choice
Double startup (site profile and user profile both fire)	"loaded"	Yes	Idempotency guard skips re-initialization

3.4 RENV PROJECT SCOPING: THE RENV_PROJECT REDIRECTION

renv is **project-scoped by design**: at startup it auto-detects a project root from the working directory and binds the session to that project's private library. That default is correct for conventional R projects and wrong for us — a user opening a file under `Programs/Table/` would silently get a new, empty per-folder renv project instead of the central library. With 63 milestone folders, per-folder libraries would mean 63 package installations to build, lock, and keep consistent.

Our redirection is one line, executed in `.Rprofile` *before* renv activation:

```
Sys.setenv(RENV_PROJECT = "<R_Config directory>")    # before
source("<R_Config>/renv/activate.R")
```

Every session on the server then resolves to the single central renv project regardless of working directory.

This usage requires an explicit warning. We pin **renv 1.2.3** in production (the documentation check below was run against the then-current release, 1.2.4). As verified on 2026-09-25 against the renv 1.2.4 documentation — specifically the configuration reference, which documents the `RENV_CONFIG_*` family, and the FAQ — `RENV_PROJECT` is **not documented as a supported project-redirection mechanism**. It is an undocumented, unsupported usage of renv with known failure modes when renv updates its bootstrap logic: the generated `activate.R` may change how it resolves the project, and the redirection could break silently on a renv upgrade. Our mitigations are the version pin itself, the `test_rconfig.R` verification suite (Appendix A) run after any renv change, and the escape hatch below. Teams adopting this pattern should treat it as a temporary bridge and re-verify it on every renv upgrade, or migrate to a supported mechanism (e.g., Posit Package Manager snapshots, Section 2.5) when infrastructure allows.

The central-versus-project tension is governed by an explicit policy: **central by default, escape hatch explicit**. The central library is located via `RENV_PATHS_LIBRARY`; a bootstrap re-source in `rconfig.R` overrides any stale library paths left by an earlier partial activation; and a study that genuinely needs isolation may place its own `.Rprofile` in its milestone folder, which R's startup order will honor over the chain — a documented, visible, per-study decision rather than an accident.

3.5 VERSIONED CENTRAL LIBRARIES AND PER-STUDY PINNING: A WORKED TRACE

Centralization creates an apparent contradiction with reproducibility: if all studies resolve to one central library, how does a completed study stay on library v1 after the team moves to v2? The answer is **append-only versioned libraries plus an explicit pointer**:

Libraries live in versioned directories, e.g. `renv/library/v1/`, `renv/library/v2/`. New versions are added; old ones are never modified.

A global pointer (an environment variable set in the central `.Renviron`) names the *current* version for new work.

A study that must re-run on an older library pins it via the escape hatch: a one-line milestone-level `.Rprofile` setting `RENV_PATHS_LIBRARY` to the pinned version, plus its archived `renv.lock` documenting the exact package set.

Worked trace. Study A completed on library v1 (R 4.5.x, dplyr 1.1.4, lme4 1.1-38). The team later advances the global pointer to v2 (updated dplyr and friends). An un-pinned study B session now sees:

```
# Study B — new work, follows the global pointer (v2)
.libPaths()
# [1] "<R_Config>/renv/library/v2/R-4.5/x86_64-w64-mingw32"
# [2] "<R installation>/library"
```

When a re-run of Study A is required (e.g., a regulatory request), its milestone folder carries the explicit pin:

```
# <Study A>/<milestone>/.Rprofile — the documented escape hatch
Sys.setenv(RENV_PATHS_LIBRARY = "<R_Config>/renv/library/v1")
```

and the re-run session confirms the resolution:

```
# Study A — re-run on the pinned library (v1), while the global pointer is
v2
.libPaths()
# [1] "<R_Config>/renv/library/v1/R-4.5/x86_64-w64-mingw32"
# [2] "<R installation>/library"
```

The archived lockfile for Study A records the package set the re-run must reproduce:

```
{
  "R": { "Version": "4.5.1" },
  "Packages": {
    "dplyr":  { "Version": "1.1.4"  },
    "lme4":   { "Version": "1.1-38" },
    "admiral": { "Version": "1.x" }
  }
}
```

This resolves the centralization-versus-reproducibility tension operationally: one physical library location per version, one global default for new work, per-study pins where the audit trail requires them.

3.6 PACKAGE MANAGEMENT

The central library holds **70 packages** (Appendix B), organized by clinical function and loaded type-aware: `.Rprofile` infers the program type from the folder name (`Programs/Table/` → table packages; `Programs/Figure/` → visualization packages) and loads the core set plus the relevant category, reducing startup time and namespace pressure. The category structure below is the same enumeration as Appendix B:

Category	Packages	Examples
Core (always loaded)	10	conflicted, haven, dplyr, tidyr, stringr, lubridate
Data I/O	6	readxl, openxlsx, data.table, arrow, jsonlite
Data cleaning	5	janitor, scales, labelled
Clinical / CDISC	2	admiral, xportr
Tables & reporting	12	rtables, r2rtf, flextable, gt, gtsummary, sassy
Visualization	14	ggplot2, ggsurvfit, patchwork, swimplot
Modeling & statistics	14	survival, lme4, lmerTest, mmrm, rpact

Category	Packages	Examples
Tools	6	shiny, devtools, fs, here, rstudioapi
Meta	1	tidyverse
Total	**70**	

3.7 NAMESPACE CONFLICT RESOLUTION

R's namespace conflicts are a real correctness hazard in clinical programming: `stats::filter()` silently shadowing `dplyr::filter()` changes results, not just messages. `.Rprofile` therefore loads the conflicted package and declares explicit winners for the common collisions (`filter`, `lag`, `select`, and others), so that an ambiguous call fails loudly instead of resolving silently. The declarations live in the central `.Rprofile` (Appendix A), which makes conflict policy a team decision applied uniformly, rather than each programmer's private habit.

3.8 BATCH SUBMISSION INTEGRATION

The studio's legacy batch scheduler submits R programs through a VBScript wrapper. We updated it to inject environment variables — the program file path and pointers to the central `.Rprofile` and `.Renviron` — before invoking `Rscript`, which is what makes batch sessions identical to interactive ones (Section 2.4).

One honest caveat: Microsoft has announced the deprecation of VBScript, with removal from future Windows releases planned in phases. Our wrapper works today, but the successor path is already identified: the same four environment-variable assignments and the `Rscript` call translate mechanically into a PowerShell wrapper (`sync_r_config.ps1` in Appendix A shows the team's existing PowerShell precedent), and the migration is a scheduler-side change that R programs will never see, precisely because environment injection — not VBScript itself — is the mechanism.

---

4. GXP-COMPATIBLE CONTROLS

4.1 VERSION LOCKING

The central renv project maintains a `renv.lock` capturing the exact version of every package in the library, and the R version itself. Beyond the lockfile, `init_renv.R` enforces explicit pins for packages whose behavior is output-relevant: for example, lme4 version 2.0 changed formula-handling behavior relevant to mixed-effects models, so the initialization script detects any newer installation and restores the pinned 1.1-38. Locks plus active pinning, not locks alone, are what make "the same versions" a property of the environment rather than a hope.

4.2 R VERSION ENFORCEMENT

All users on the server share a single R installation, deliberately mirroring the SAS model in which one validated SAS version serves the whole programming group. `.Rprofile` checks the running R version against the required major.minor at every startup and emits a prominent warning on mismatch, so a stray second installation (e.g., a user-installed R on a local drive) is caught at session start rather than at output-review time.

4.3 AUDIT TRAIL ARCHITECTURE

What we claim: every program execution leaves four artifacts — the `bps` object documenting every derived path for that run; the `renv.lock` documenting exact package versions; the `.Rout` log capturing the full batch session; and the studio's own audit trail recording who ran what and when. Together these make any output traceable to the code, data snapshot, package set, and R version that produced it.

What we do *not* claim: this is not a validated audit-trail system in the 21 CFR Part 11 sense. There are no electronic signatures, no tamper-proof logging, and no independent verification service. The audit-trail support described here is a foundation that an organization's validated processes can build on, not a substitute for them.

4.4 VALIDATION-SCOPE STATEMENT

This section deserves precision. The mechanisms in this paper — version locking, R version enforcement, append-only libraries, startup verification — are **controls that support operation in a GxP context. They are not, and we do not present them as, completed system validation.** No IQ/OQ qualification of the environment is claimed here, and no computer-system-assurance evidence package is asserted.

This framing follows the risk-based thinking of the R Validation Hub white paper, which assesses R *packages* by risk within a validated infrastructure rather than treating "validation" as a monolithic property of R itself, and is consistent with GAMP5-style categorization, under which the R runtime and standard packages behave as infrastructure-like components while the team's configuration layer is the configured, risk-assessed element. The lock itself was documented practice: a dated environment-lock record (2026-06-08) captured the R version, the controlling environment variables, the R options, and the lockfile package versions, and was marked validated at lock time. Our configuration layer is small, inventory-documented (Appendix A), and covered by its own verification suite (`test_rconfig.R`) precisely so that an organization's validation process has a bounded, inspectable surface to assess. What we have built is the controllable environment a validation exercise needs; the validation itself remains the organization's responsibility.

4.5 SUPPRESSWARNINGS FOR CROSS-VERSION BINARIES: BOUNDED JUSTIFICATION

One control requires a caveat rather than a celebration. When CRAN Windows binaries are compiled against a newer R patch version than the server's (e.g., binaries built under 4.5.3 loading under R 4.5.1), R emits a benign warning on every `library()` call. To programmers new to R, a screen of warnings at every startup reads as a broken environment, and it trains them to ignore warnings generally — the worst possible outcome in a GxP context. `.Rprofile` therefore wraps only the startup `library()` calls in `suppressWarnings()`.

The justification is deliberately bounded: suppression applies exclusively to package attachment at startup, never to analysis code; the suppressed messages are of one known, benign class (binary built under a newer patch of the same minor version); and the `test_rconfig.R` suite exercises the library so that a genuinely incompatible binary — the residual risk that suppression hides — would surface as an error at verification time rather than a warning at use time. The residual risk is real but narrow: a binary incompatibility that produced wrong results *without* error would evade both layers. We judge that risk acceptable for patch-level skew within one minor R version, and unacceptable across minor versions, which is why Section 4.2 forbids them.

---

5. RESULTS

5.1 DEPLOYMENT PROCESS

Deployment is deliberately boring:

Step	Who	Effort	Frequency
Create `Rprofile.site` (3 lines)	IT admin	~5 min	One-time
Initialize the renv library	R lead	~20 min	One-time

Step	Who	Effort	Frequency
Deploy updated R_Config files	R lead	~5 min	Per configuration update
Run `test_rconfig.R` verification	R lead	minutes	After every update
New-user onboarding	—	No user action	—

The last row is the point of the paper: after the one-time IT action, a new team member's first R session on the server simply works.

The measurements in this paper were collected on the production server on the dates noted (September 2026). The system presumes four conditions, all common in SAS-centric studios: (i) one shared server that all programmers use; (ii) IT able to perform a single three-line site-file install; (iii) a team-writable configuration directory on the same network; and (iv) a batch scheduler that can set environment variables — any scheduler, since injection, not VBScript, is the mechanism (Section 3.8). Where these hold, no per-user action is ever required; where they do not — no administrator access at all, or container infrastructure already available — Section 2.5 maps the better-tooled alternatives.

5.2 BEFORE/AFTER COMPARISON

Table 2 summarizes the environment-setup changes, before and after the centralized system.

Table 2. Environment setup and consistency, before and after.

Metric	Before	After
New-user environment setup	Documented 30–60 min checklist: install packages, configure paths, verify versions	Single automatic startup; no user action
Config files per milestone folder	2–3 per folder	0 (centralized)
Package version consistency	User-installed; versions varied across users	One shared, version-pinned library
Path configuration	Manual, per program	Auto-detected from program location
Namespace conflict handling	Ad hoc, per programmer	Centralized and enforced
Batch vs. interactive parity	Different environments	Identical, via injected environment variables

5.3 USER EXPERIENCE: THE STARTUP BANNER

When a programmer opens any R file under the workspace, the session announces itself:

```
======================================================
  Studio R Environment Ready
------------------------------------------------------
  R Version:    R version 4.5.1 (2025-06-13 ucrt)
  Program Type: TABLE
------------------------------------------------------
  Compound:     <Compound>
  Study:        <Study>
  Milestone:    <Milestone>
------------------------------------------------------
  Raw Data:     <shared network location>/.../<Milestone>/Data/Raw
  SDTM Data:    <shared network location>/.../<Milestone>/Data/SDTM
  ADaM Data:    <shared network location>/.../<Milestone>/Data/ADaM
```

```
  Output:       <shared network location>/.../<Milestone>/Output
-------------------------------------------------------
  renv:          active (package versions locked)
=======================================================
```

The banner is not cosmetic. It converts three silent failure classes — wrong folder assumption, wrong R version, unlocked library — into visible, checkable facts at the start of every session. (The rendering above is the genericized form; the production banner was captured live on the server during the 2026-09-26 evidence-collection runs — program type detected as CORE, locked renv library active — corroborating the type-aware loading of Section 3.6.)

5.4 CROSS-COMPOUND DEPLOYMENT

The deployment scale is measured, not asserted: a folder enumeration of the production server on 2026-09-28 (directory counts only; no data files were read) found **five production compounds, 25 study folders, and 63 milestone folders with program directories**, all sharing the single version-pinned library. Operationally, cross-compound rollout cost nothing per study: because configuration is central and detection is positional, a new compound or study folder inherits the full environment the moment its first program is opened.

5.5 VERIFICATION EVIDENCE AND MEASUREMENT BASIS

We commit to a specific measurement framework for the headline onboarding claim, so that the numbers we report are defensible rather than anecdotal:

Setup task definition. “Before” = completing the documented per-user setup checklist (install R packages, configure paths, verify versions) on a fresh account; “After” = opening any R program under the workspace and reaching the ready banner.

Sample. N onboarded users, drawn from onboarding records.

Measurements. Before/after wall-clock timings per user, and setup-related support contacts per month over comparable windows.

Pass criteria. After-state sessions reach the ready banner with the pinned library and correct derived paths, verified by `test_rconfig.R`, in 100% of sampled onboardings; no per-user configuration steps required.

One component of the after-state is measurable today, and we measured it: the per-session cost of the automatic startup itself. On the production Windows server (single enforced R version 4.5.1, locked renv library active), thirty timed R process launches with the full environment chain took a mean of 2.89 s (sd 0.08) to first executable statement, versus 0.21 s (sd 0.01) for --vanilla launches: the automatic startup — locked-library activation, package loading, guard installation, banner — adds ≈2.7 s once per session. A companion development-host run (R 4.6.0, N = 10) measured the same chain within run-to-run noise of vanilla (0.37 s vs 0.41 s), indicating the cost is dominated by the locked-library activation on the server rather than by the bootstrap logic itself. The measurement scripts and raw timing logs are retained by the authors (collected 2026-09-26). Even at the server value, the after-state is “the ordinary act of starting R, plus about three seconds” — against the documented 30–60 minute checklist it replaces. The checklist side of the comparison awaits the manual protocol.

We state the after-state as a **single automatic startup**, not an unqualified “zero setup time”: the checklist is eliminated, and what remains is the ordinary act of starting R, which the programmer was going to do anyway.

---

6. LESSONS LEARNED AND LIMITATIONS

6.1 RENV IS PROJECT-SCOPED BY DEFAULT (AND THE FIX)

The deepest assumption we fought was renv’s: one project, one library, project root inferred from the working directory. Our architecture wants the opposite — one library, 63 working directories. The

`RENV_PROJECT` redirection (Section 3.4) resolved it, but only as a pinned, undocumented, unsupported usage that we must re-verify on every renv upgrade. Teams should budget for that maintenance or adopt a supported central-library mechanism where infrastructure permits.

6.2 RM(LIST=LS()) IS PERVASIVE (AND THE GUARD)

Legacy templates, textbooks, and training materials all teach `rm(list=ls())`. Infrastructure that breaks when users follow common advice is fragile infrastructure. The dual-check guard (Section 3.3) made the environment survive its users' habits; new templates now avoid the blanket `rm()` in favor of targeted cleanup, but the guard stays, because old habits outlive templates.

6.3 ADMIN ACCESS IS OFTEN UNAVAILABLE (AND THE MINIMAL-TOUCH PATTERN)

Clinical programming teams rarely hold admin rights on production servers, and every admin touchpoint is a ticket, a queue, and a change-control entry. Concentrating all IT responsibility into one three-line site file, installed once, was the single most consequential design decision: everything afterward is team-managed in a writable directory. Design R infrastructure to minimize admin touchpoints; use `Rprofile.site` as a permanent pointer, never as a working surface.

6.4 R'S STARTUP ORDER MATTERS

R's startup sequence — site profile, environment files, user profile — and the interaction of `R_PROFILE_USER` and `R_ENVIRON_USER` with site-level configuration is documented but subtle, and real deployments hit the subtle parts immediately: double execution when both site and user profiles fire, environment variables that exist in batch but not interactive sessions, working directories that differ between contexts. Idempotency guards and explicit path detection exist in our code because each of these bit us once.

6.5 ENVIRONMENT GOVERNANCE MUST BE EXPLICIT

A shared central library is a shared fate. Who may bump a package version, when the global pointer advances, which studies may pin, and who verifies after a change — if these are not written down, the central library drifts into the same inconsistency it was built to prevent. Our rule set is short: central by default, escape hatch explicit, append-only libraries, verification suite mandatory after every change.

6.6 LIMITATIONS

We state these plainly:

Single-server scope. The system runs on one shared Windows server. Multi-server, multi-site, or cloud deployments would need additional replication and governance that we have not built or tested.

Windows and legacy-stack specifics. Path normalization, the VBScript batch wrapper, and several failure modes are specific to our Windows/legacy-scheduler stack; the patterns transfer, the code does not, unchanged.

Unsupported renv usage. The `RENV_PROJECT` redirection is undocumented and unsupported (Section 3.4), pinned to renv 1.2.3, and a known maintenance liability on renv upgrades.

Validation evidence not claimed. We present GxP-compatible controls, not completed validation (Section 4.4); organizations must run their own risk assessment and qualification.

Specific operating conditions. The system assumes the four conditions of Section 5.1 — one shared server, a one-time IT site-file install, a team-writable configuration directory, and environment-injecting scheduling. These are common but not universal; the pattern does not claim to cover organizations outside them.

Onboarding measurements pending. The 30–60 minute checklist is documented, but the measured before/after timings and user counts (Section 5.5) await prospective collection via the prepared measurement protocol; a document search on 2026-09-28 found department-level onboarding materials but no R-environment-specific timing records, and no support-ticket system.

7. GENERALIZABILITY

The architecture transfers to any organization that shares our corner of the design space: a shared server (Windows or Linux) for clinical programming, an existing SAS infrastructure with standardized folder conventions, a desire to add R without disrupting SAS workflows, limited administrator access, and GxP-compatible control requirements. The transferable abstraction is the three-layer bootstrap — site profile → configuration script → environment profile — which cleanly separates IT responsibility (one-time, three lines), team responsibility (the configuration directory), and user responsibility (none). Around that spine, the pieces are ordinary: one path-detection cascade, one `setup()` that walks a folder tree, one guard against `rm(list=ls())`, one explicit governance rule set.

8. CONCLUSION

The result, in daily use, is that R in our studio now feels like SAS does: open the program, and the environment is already right. That is the whole ambition of this paper — and it is only the foundation. What a team can build once the environment is invisible — one code path producing submission-ready TLFs in both batch production and interactive preview, reading the same metadata the SAS programs read — is the subject of our companion paper, "One Code, Two Modes: A Convention-Driven R Reporting System for Submission-Ready Clinical TLFs" (same authors, SEASUG 2026).

REFERENCES

Zhang, J., and Cook, J. 2025. "Building Robust R Workflows: renv for Version Control and Environment Reproducibility." *Proceedings of the PharmaSUG 2025 Conference*, Paper SD-220.

Bowsher, P., Lopp, S., and Gold, A. 2021. "R Reproducibility Best Practices for Pharma." *Proceedings of the PharmaSUG 2021 Conference*, Paper AP-204. https://pharmasug.org/proceedings/2021/AP/PharmaSUG-2021-AP-204.pdf

Ushey, K., and Wickham, H. *renv: Project Environments for R*. R package version 1.2.4 documentation (deployed version pinned at 1.2.3), including the configuration reference (`?renv::config`, `RENV_CONFIG_*` variables) and FAQ. https://rstudio.github.io/renv/ (accessed 2026-09-25).

R Core Team. *R Installation and Administration*. Section "Initialization at Start of an R Session." https://cran.r-project.org/doc/manuals/r-release/R-admin.html

pharmaverse. *envsetup: Setup Development Environment*. R package version 0.3.0. https://pharmaverse.github.io/envsetup/ (accessed 2026-09-26; Section 2.5 feature table verified against this release).

Posit. *Posit Package Manager Documentation: CRAN Snapshots and Dated Repositories*. https://docs.posit.co/rspm/

R Validation Hub. *A Risk-Based Approach for Assessing R Package Accuracy within a Validated Infrastructure* (white paper). https://www.pharmar.org/white-paper/

U.S. Food and Drug Administration. 2015. *Statistical Software Clarifying Statement*. "FDA does not require use of any specific software for statistical analyses."

Wickham, H. *conflicted: An Alternative Conflict Resolution Strategy*. R package version 1.2.0. https://conflicted.r-lib.org/

Lee, K., and Jeeva, M. 2022. "Enterprise-level Transition from SAS® to Open-Source Programming for the Whole Department." *Proceedings of the PharmaSUG 2022 Conference*, Paper SI-044. https://pharmasug.org/proceedings/2022/SI/PharmaSUG-2022-SI-044.pdf

Palukuru, U. P., Li, R., Patel, N., and Shi, C. 2022. "Exploring R as a Validation Tool for Statistical Programming in Clinical Trials." *Proceedings of the PharmaSUG 2022 Conference*, Paper AD-076. https://pharmasug.org/proceedings/2022/AD/PharmaSUG-2022-AD-076.pdf

Metrum Research Group. *pkgr: Install R Packages from a Lockfile with System-Command Control*. https://github.com/metrumresearchgroup/pkgr (accessed 2026-09-26); and Csárdi, G. *rig: R Installation Manager*. https://github.com/r-lib/rig (accessed 2026-09-26). Cited in the Section 2.5 comparison landscape.

APPENDIX A: CONFIGURATION-DIRECTORY FILE INVENTORY

The following is the author's inventory of the server-side configuration artifacts (`Rprofile.site`, the `R_Config/` directory, and the renv library live on the shared server; the inventory is reported as maintained by the programming team and was not independently re-verified for this paper).

File	Lines	Purpose
`Rprofile.site`	3	IT-managed site profile; permanent pointer to `rconfig.R` (Section 2.3)
`rconfig.R`	462	Path-detection cascade, `setup()`, library helpers (Sections 3.1–3.3)
`.Rprofile`	425	renv activation, package loading, conflict policy, banner (Sections 3.4–3.7)
`.Renviron`	26	Environment variables for all studio R sessions
`init_renv.R`	130	One-time library initialization and critical-package pinning (Section 4.1)
`test_rconfig.R`	108	Verification suite, run after every configuration change
`sync_r_config.ps1`	—	PowerShell sync with the development environment (Section 3.8 precedent)
`rBatchSubmit.vbs`	119	Batch wrapper with environment injection (Section 3.8; VBScript deprecated — PowerShell successor path identified)
`renv/activate.R`, `renv/library/v*/`	—	renv bootstrap and append-only versioned libraries (Sections 3.4–3.5)

APPENDIX B: PACKAGE REGISTRY (70 PACKAGES)

Core (always loaded, 10): conflicted, haven, dplyr, tidyr, stringr, lubridate, rlang, purrr, tibble, forcats

Data I/O (6): readxl, openxlsx, writexl, data.table, arrow, jsonlite

Data Cleaning (5): janitor, scales, labelled, skimr, hms

Clinical / CDISC (2): admiral, xportr

Tables & Reporting (12): rtables, formatters, r2rtf, rlistings, flextable, officer, gt, gtsummary, DT, knitr, rmarkdown, sassy

Visualization (14): ggplot2, ggsurvfit, cowplot, patchwork, gridExtra, ggpubr, ggrepel, ggpmisc, ggeffects, RColorBrewer, fmsb, plotly, swimplot, corrplot

Modeling & Statistics (14): survival, lme4, lmerTest, mmrm, MASS, glmnet, randomForest, xgboost, e1071, modelsummary, modelr, rstatix, MuMIn, rpact

Tools (6): shiny, devtools, remotes, fs, here, rstudioapi

Meta (1): tidyverse

Total: 70 packages (10 + 6 + 5 + 2 + 12 + 14 + 14 + 6 + 1).

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the authors at:

Jaime Yan
Kardigan
Email: jayan@kardigan.bio
George Wu
Kardigan
Email: gwu@kardigan.bio
Jessie Yan
Kardigan
Email: jyan@kardigan.bio